

申望儿童关爱中心 AI 助教技术方案

1. 项目目标

开发一个面向儿童课后学习场景的 AI 助教，提供：

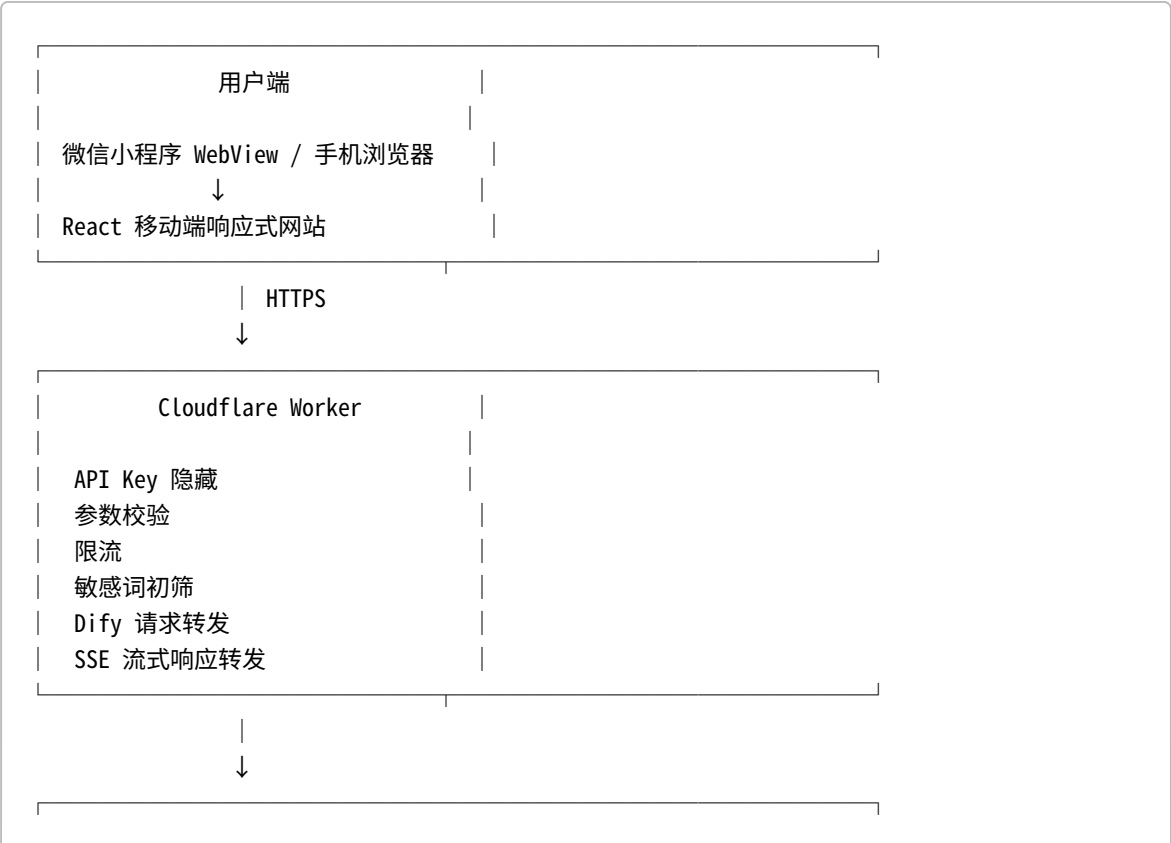
- 1. 文字提问与连续对话；
- 2. 拍照识题；
- 3. 分步骤讲解；
- 4. 教材与中心资料检索；
- 5. 相似题生成；
- 6. 常见问题推荐；
- 7. 教师端知识库维护与基础统计；
- 8. 敏感内容拦截和人工求助入口。

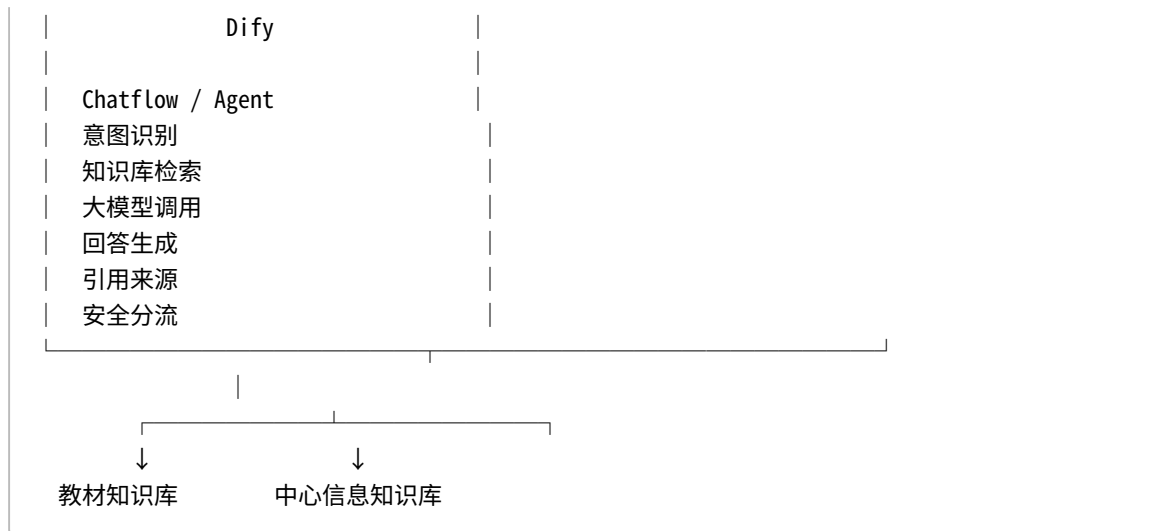
项目定位应为：

利用大模型、Dify 工作流和轻量知识库，为儿童提供基础课后答疑，并帮助教师减少重复性解释工作。

不要宣称“替代教师”“自动完成教学”或“能够解决所有学科问题”。

2. 总体技术架构





2.1 各层职责

React 前端

负责：

- 页面和动画；
- 用户输入；
- 图片选择与预览；
- 流式显示回答；
- 本地保存会话编号；
- 展示引用来源；
- 用户反馈；
- 网络错误提示。

Cloudflare Worker

负责：

- 保存 Dify API Key；
- 转发 Dify API；
- 阻止浏览器直接访问 Dify；
- 设置跨域策略；
- 验证请求长度和类型；
- 基础限流；
- 日志脱敏；
- 后期接入用户系统。

Dify 官方明确要求应用 API Key 只应在后端使用，不应嵌入浏览器或客户端代码。Dify Cloud 的应用 API 默认基地址为 `https://api.dify.ai/v1`。

Dify

负责：

- 对话上下文；

- 知识库 RAG；
- 意图路由；
- Agent 或工作流；
- 模型调用；
- Prompt；
- 输出格式；
- 引用来源；
- 会话管理。

微信小程序

第一版只负责：

- 提供微信内入口；
- 使用 `web-view` 打开响应式网站；
- 后续可增加微信登录、分享、订阅消息。

3. 推荐工具清单

环节	工具	用途
代码编辑	Visual Studio Code	编写前端和 Worker
版本管理	Git、GitHub	保存代码、协作、回滚
前端框架	React 19、TypeScript	页面与交互
构建工具	Vite	本地开发和生产构建
UI 样式	Tailwind CSS	快速完成响应式页面
图标	Lucide React	统一图标
动画	Motion	页面和消息动画
Markdown	react-markdown	渲染模型回答
数学公式	KaTeX、remark-math	渲染数学公式
状态管理	Zustand	会话和用户状态
请求层	原生 Fetch API	流式请求
AI 后端	Dify Cloud	Agent、Chatflow、RAG
API 代理	Cloudflare Workers	隐藏密钥、转发流
静态部署	Cloudflare Pages	发布网站
小程序工具	微信开发者工具	开发小程序壳
接口测试	Bruno 或 Postman	测试 Worker 和 Dify
设计	Figma	页面原型
文档整理	Word、Markdown	准备知识库资料

第一期不建议引入：

- Kubernetes;
- LangChain 自定义后端;
- 微服务;
- Redis 集群;
- 独立向量数据库;
- 原生 Android;
- 多 Agent 协作;
- 本地部署大模型。

这些技术会显著增加部署和排错成本，但不会明显改善演示效果。

4. 项目目录

建议建立一个 Git 仓库：

```
shenwang-ai-tutor/
├── web/                                # React 响应式网站
│   ├── src/
│   │   ├── components/
│   │   │   ├── ChatMessage.tsx
│   │   │   ├── ChatInput.tsx
│   │   │   ├── SourceCard.tsx
│   │   │   ├── QuickQuestions.tsx
│   │   │   └── LoadingMessage.tsx
│   │   ├── pages/
│   │   │   ├── HomePage.tsx
│   │   │   ├── ChatPage.tsx
│   │   │   ├── HistoryPage.tsx
│   │   │   └── AboutPage.tsx
│   │   ├── services/
│   │   │   └── chat.ts
│   │   ├── stores/
│   │   │   └── chatStore.ts
│   │   ├── types/
│   │   │   └── chat.ts
│   │   └── main.tsx
│   ├── public/
│   ├── package.json
│   └── vite.config.ts
├── worker/                             # Cloudflare Worker
│   ├── src/
│   │   └── index.ts
│   ├── wrangler.jsonc
│   └── package.json
└──
```

```
├─ miniprogram/                # 微信小程序外壳
│  ├─ pages/
│  │   └─ index/
│  ├─ app.json
│  └─ project.config.json
│
├─ knowledge/                  # 待入库资料
│  ├─ center/
│  ├─ math-grade5/
│  └─ manifests/
│
├─ docs/
│  ├─ architecture.md
│  ├─ test-cases.xlsx
│  └─ privacy-design.md
│
└─ README.md
```

5. 第一阶段：创建 React 响应式网站

5.1 安装环境

安装：

- Node.js 当前 LTS 版本；
- Git；
- Visual Studio Code。

在终端检查：

```
node -v
npm -v
git --version
```

5.2 创建项目

```
mkdir shenwang-ai-tutor
cd shenwang-ai-tutor

npm create vite@latest web -- --template react-ts
cd web
npm install
npm run dev
```

浏览器打开终端显示的本地地址。

5.3 安装常用依赖

```
npm install react-router-dom zustand
npm install react-markdown remark-gfm remark-math rehype-katex katex
npm install lucide-react motion
```

安装 Tailwind 时，应按当前 Tailwind 与 Vite 官方流程配置。不要照搬几年前使用 `tailwind.config.js` 的旧教程而不对版本。

5.4 定义消息类型

`src/types/chat.ts` :

```
export type MessageRole = "user" | "assistant" | "system";

export interface SourceReference {
  title: string;
  page?: string;
  content?: string;
  url?: string;
}

export interface ChatMessage {
  id: string;
  role: MessageRole;
  content: string;
  createdAt: number;
  status?: "sending" | "streaming" | "done" | "error";
  sources?: SourceReference[];
}
```

5.5 会话状态

`src/stores/chatStore.ts` :

```
import { create } from "zustand";
import type { ChatMessage } from "../types/chat";

interface ChatState {
  messages: ChatMessage[];
  conversationId: string;
  isGenerating: boolean;

  addMessage: (message: ChatMessage) => void;
  updateMessage: (id: string, patch: Partial<ChatMessage>) => void;
  setConversationId: (id: string) => void;
  setGenerating: (value: boolean) => void;
}
```

```

clearConversation: () => void;
}

export const useChatStore = create<ChatState>((set) => ({
  messages: [],
  conversationId:
    window.localStorage.getItem("conversation_id") ?? "",
  isGenerating: false,

  addMessage: (message) =>
    set((state) => ({
      messages: [...state.messages, message],
    })),

  updateMessage: (id, patch) =>
    set((state) => ({
      messages: state.messages.map((item) =>
        item.id === id ? { ...item, ...patch } : item
      ),
    })),

  setConversationId: (id) => {
    window.localStorage.setItem("conversation_id", id);
    set({ conversationId: id });
  },

  setGenerating: (value) => set({ isGenerating: value }),

  clearConversation: () => {
    window.localStorage.removeItem("conversation_id");
    set({ messages: [], conversationId: "" });
  },
}));

```

5.6 页面结构

移动端建议采用：

顶部栏

- └─ Logo
- └─ AI 助教名称
- └─ 新建会话

欢迎区

- └─ 人物插画
- └─ “今天想学习什么？”
- └─ 年级/学科标签

快捷功能

- └─ 拍照解题
- └─ 数学答疑
- └─ 课文讲解
- └─ 中心信息

消息区域

- └─ 用户消息
- └─ AI 消息
- └─ 来源卡片
- └─ 反馈按钮

底部输入框

- └─ 图片按钮
- └─ 文本输入
- └─ 发送按钮

5.7 响应式原则

主要设计宽度：

- 375 px：普通手机；
- 430 px：大屏手机；
- 768 px：平板；
- 1280 px：网页演示。

建议内容区：

```
.chat-container {  
  width: min(100%, 760px);  
  margin: 0 auto;  
}
```

避免桌面网页直接缩小到手机。按钮触摸区域建议保持足够大，正文不要使用过小字号。

6. 第二阶段：建立 Cloudflare Worker 代理

6.1 为什么必须有代理

错误结构：

```
React → Dify API
```

此时 Dify API Key 会被写进前端 JavaScript，任何人都能从浏览器开发者工具中提取。

正确结构：


```
React → Worker → Dify API
```

Dify 官方文档明确说明，API Key 应保存在服务端。

6.2 创建 Worker

回到项目根目录：

```
cd ..  
npm create cloudflare@latest worker
```

选择：

```
Hello World example  
Worker only  
TypeScript  
Git: Yes  
Deploy now: No
```

进入目录：

```
cd worker  
npm install  
npm run dev
```

6.3 配置文件

`wrangler.jsonc`：

```
{  
  "$schema": "node_modules/wrangler/config-schema.json",  
  "name": "shenwang-ai-proxy",  
  "main": "src/index.ts",  
  "compatibility_date": "2026-07-01",  
  "vars": {  
    "DIFY_BASE_URL": "https://api.dify.ai/v1",  
    "ALLOWED_ORIGIN": "http://localhost:5173"  
  }  
}
```

普通配置可以放在 `vars` 中，但 API Token 应使用 Worker Secret。Cloudflare 将 secret 用于密码和 API Token 等敏感数据，并且保存后不会在控制台中再次显示其值。

6.4 添加 Dify Secret

```
npx wrangler secret put DIFY_API_KEY
```

输入 Dify 应用 API Key。

不要把 Key 放进：

```
.env 提交到 GitHub  
wrangler.jsonc  
前端 VITE_* 变量  
React 源码  
小程序源码
```

6.5 Worker 基础代码

worker/src/index.ts :

```
interface Env {  
  DIFY_API_KEY: string;  
  DIFY_BASE_URL: string;  
  ALLOWED_ORIGIN: string;  
}  
  
interface ChatRequest {  
  query: string;  
  conversationId?: string;  
  userId?: string;  
  grade?: string;  
  subject?: string;  
}  
  
function corsHeaders(origin: string): HeadersInit {  
  return {  
    "Access-Control-Allow-Origin": origin,  
    "Access-Control-Allow-Methods": "POST,OPTIONS",  
    "Access-Control-Allow-Headers": "Content-Type",  
    "Access-Control-Expose-Headers": "Content-Type",  
    "Vary": "Origin",  
  };  
}  
  
function jsonResponse(  
  body: unknown,  
  status: number,  
  origin: string  
): Response {
```

```

return new Response(JSON.stringify(body), {
  status,
  headers: {
    "Content-Type": "application/json; charset=utf-8",
    ...corsHeaders(origin),
  },
});
}

export default {
  async fetch(
    request: Request,
    env: Env
  ): Promise<Response> {
    const requestOrigin = request.headers.get("Origin") ?? "";
    const allowedOrigin = env.ALLOWED_ORIGIN;

    if (request.method === "OPTIONS") {
      return new Response(null, {
        status: 204,
        headers: corsHeaders(allowedOrigin),
      });
    }

    if (requestOrigin && requestOrigin !== allowedOrigin) {
      return jsonResponse(
        { error: "Origin not allowed" },
        403,
        allowedOrigin
      );
    }

    const url = new URL(request.url);

    if (url.pathname !== "/api/chat") {
      return jsonResponse(
        { error: "Not found" },
        404,
        allowedOrigin
      );
    }

    if (request.method !== "POST") {
      return jsonResponse(
        { error: "Method not allowed" },
        405,
        allowedOrigin
      );
    }

    let body: ChatRequest;

```

```

try {
  body = await request.json();
} catch {
  return jsonResponse(
    { error: "Invalid JSON" },
    400,
    allowedOrigin
  );
}

const query = body.query?.trim();

if (!query) {
  return jsonResponse(
    { error: "Question is required" },
    400,
    allowedOrigin
  );
}

if (query.length > 2000) {
  return jsonResponse(
    { error: "Question is too long" },
    413,
    allowedOrigin
  );
}

const userId =
  body.userId?.trim() ||
  crypto.randomUUID();

const difyResponse = await fetch(
  `${env.DIFY_BASE_URL}/chat-messages`,
  {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${env.DIFY_API_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      inputs: {
        grade: body.grade ?? "",
        subject: body.subject ?? "",
      },
      query,
      response_mode: "streaming",
      conversation_id:
        body.conversationId ?? "",
      user: userId,
    })
  )
}

```

```

        auto_generate_name: true,
    }},
    }
);

if (!difyResponse.ok || !difyResponse.body) {
    const errorText = await difyResponse.text();

    console.error("Dify request failed", {
        status: difyResponse.status,
        error: errorText.slice(0, 500),
    });

    return jsonResponse(
        { error: "AI service unavailable" },
        502,
        allowedOrigin
    );
}

return new Response(difyResponse.body, {
    status: 200,
    headers: {
        "Content-Type": "text/event-stream; charset=utf-8",
        "Cache-Control": "no-cache",
        "Connection": "keep-alive",
        ...corsHeaders(allowedOrigin),
    },
});
},
};
};

```

Dify 的 `/chat-messages` 接口支持 `streaming` 和 `blocking` 两种响应模式；流式模式返回 `text/event-stream`，连续对话需要将上一次返回的 `conversation_id` 带入下一次请求。

6.6 本地测试

```
npm run dev
```

然后使用 Bruno、Postman 或 PowerShell 请求：

```

curl -N http://localhost:8787/api/chat
-H "Content-Type: application/json"
-d '{
    "query": "什么是分数的通分？",
    "userId": "test-user-001",
    "grade": "五年级",

```

```
"subject": "数学"
}'
```

能看到连续 SSE 数据，说明代理成功。

7. 第三阶段：前端读取流式回答

`web/src/services/chat.ts` :

```
export interface SendMessageInput {
  query: string;
  conversationId: string;
  userId: string;
  grade?: string;
  subject?: string;
}

export interface StreamCallbacks {
  onText: (text: string) => void;
  onConversationId?: (id: string) => void;
  onMessageId?: (id: string) => void;
  onError?: (message: string) => void;
}

const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL;

export async function sendChatMessage(
  input: SendMessageInput,
  callbacks: StreamCallbacks,
  signal?: AbortSignal
): Promise<void> {
  const response = await fetch(
    `${API_BASE_URL}/api/chat`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(input),
      signal,
    }
  );

  if (!response.ok) {
    const error = await response
      .json()
      .catch(() => null);
  }
}
```

```

throw new Error(
  error?.error ?? `HTTP ${response.status}`
);
}

if (!response.body) {
  throw new Error("服务器没有返回数据流");
}

const reader = response.body.getReader();
const decoder = new TextDecoder("utf-8");

let buffer = "";

while (true) {
  const { done, value } = await reader.read();

  if (done) break;

  buffer += decoder.decode(value, {
    stream: true,
  });

  const blocks = buffer.split("\n\n");
  buffer = blocks.pop() ?? "";

  for (const block of blocks) {
    const dataLine = block
      .split("\n")
      .find((line) => line.startsWith("data:"));

    if (!dataLine) continue;

    const raw = dataLine.slice(5).trim();

    if (!raw || raw === "[DONE]") continue;

    try {
      const event = JSON.parse(raw);

      if (
        event.event === "message" ||
        event.event === "agent_message"
      ) {
        callbacks.onText(event.answer ?? "");
      }

      if (event.conversation_id) {
        callbacks.onConversationId?.(
          event.conversation_id
        );
      }
    } catch {
      // ignore
    }
  }
}

```

```

    );
}

if(event.message_id){
  callbacks.onMessageId?.(
    event.message_id
  );
}

if(event.event === "error"){
  callbacks.onError?.(
    event.message ?? "生成失败"
  );
}
} catch {
  // 当前块不完整时留给下一轮;
  // 不应直接让整个会话崩溃。
}
}
}
}
}

```

注意：Dify 不同应用类型可能返回不同事件类型，因此开发时应把原始 SSE 事件打印出来，再针对实际使用的 Chatflow 或 Agent 做适配。Dify 文档也说明，不同应用类型的流式事件可能不同。

7.1 前端环境变量

web/.env.local :

```
VITE_API_BASE_URL=http://localhost:8787
```

这里放的是 Worker 公共地址，不是 Dify API Key，因此允许暴露。

8. 第四阶段：配置 Dify

8.1 选择 Dify Cloud 还是自部署

第一期建议：

使用 Dify Cloud，不自部署。

原因：

- 不需要维护 Docker；
- 不需要部署 PostgreSQL、Redis、向量数据库；
- 不需要处理升级和备份；

- 更适合时间有限的学生团队。

等项目真正需要长期运营或数据不能上传云端时，再评估自部署。

8.2 创建模型供应商

在 Dify 中进入模型供应商设置，配置一种主要模型和一种备用模型。

建议类型：

- 国内可稳定调用的文本模型；
- 支持视觉输入的多模态模型；
- 可选低成本快速模型用于分类。

模型选择标准：

1. 中文表达稳定；
2. 支持流式输出；
3. API 在部署地区可访问；
4. 成本可控；
5. 最好支持图片；
6. 有明确内容安全机制。

不要在第一版同时接入五六个模型。

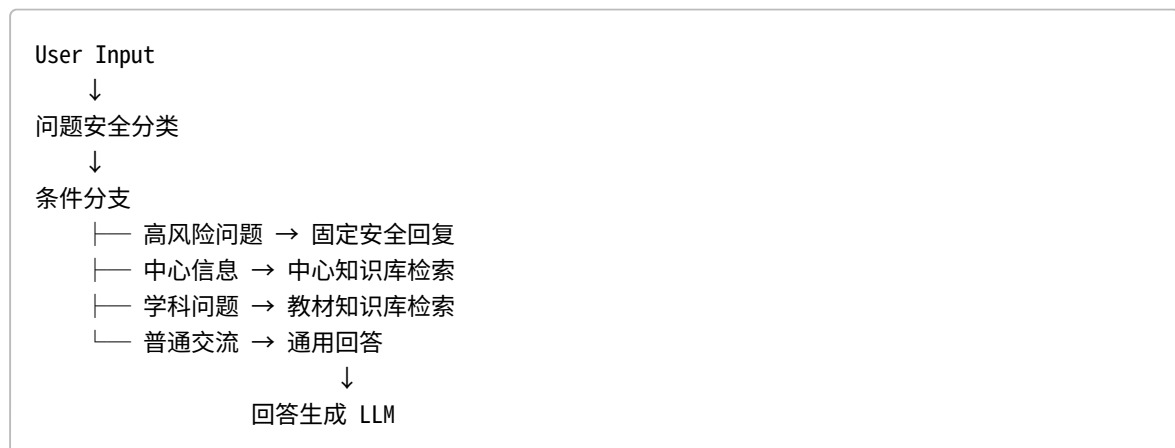
8.3 应用类型选择

推荐建立 **Chatflow**，而不是完全自由的 Agent。

原因：

- Chatflow 每轮对话都会执行；
- 支持对话变量和记忆；
- 可视化流程更稳定；
- 可以流式输出文本、图片和文件；
- 更容易控制知识库何时被调用。

第一版工作流：



↓
Answer

8.4 输入变量

在 User Input 中增加：

变量	类型	示例
<code>grade</code>	String	五年级
<code>subject</code>	String	数学
<code>query</code>	系统输入	如何通分
图片	File	题目照片

这些变量由 Worker 通过 `inputs` 传入。

8.5 意图分类

分类标签控制在四到六类：

```
study_question  
center_information  
general_chat  
safety_risk  
unsupported
```

分类 Prompt：

你是儿童学习助手的请求分类器。

根据用户问题，只输出以下一个标签：

`study_question`：
学科知识、作业、题目解析、学习方法。

`center_information`：
申望儿童关爱中心的课程、活动、时间、
教师、场地、规章等信息。

`general_chat`：
普通问候、鼓励或不需要知识库的交流。

`safety_risk`：
涉及自伤、他伤、虐待、性侵、失踪、
严重欺凌、现实危险或紧急求助。

unsupported:
系统目前无法处理的请求。

不得输出解释，不得输出其他内容。

8.6 为什么第一版不让 Agent 完全自由选择

完全 Agent 化可能出现：

- 明明需要知识库却不检索；
- 重复调用工具；
- 调错知识库；
- 输出延迟增加；
- 成本不可预测；
- 演示现场行为不稳定。

因此，第一版采用：

LLM 分类器 + 条件分支 + 知识检索节点

对外仍可称为“智能 workflow”或“Agent 化工具调用”，但内部更加可控。

9. 第五阶段：构建知识库

9.1 知识库划分

至少分成两个库，不要把所有资料混在一起。

A. 中心信息库

内容：

- 中心简介；
- 课程安排；
- 活动安排；
- 场地规则；
- 安全规定；
- 教师和志愿者职责；
- 常见问题；
- 联系方式；
- 物资使用说明。

B. 教材知识库

第一期只选：

- 一个年级；
- 一个学科；

- 一至两个单元。

示例：

五年级数学
 └─ 分数的意义
 └─ 约分
 └─ 通分
 └─ 分数加减法
 └─ 常见错误

Dify 可以将知识库作为应用的外部信息来源，也提供知识库、文档、分段和元数据相关 API。

9.2 建议规模

第一版：

内容	数量
中心资料	20~50 份
教材章节	1~2 个单元
知识点	30~60 个
典型例题	50~100 道
常见错误	20~40 条
评测问题	100 条左右

知识库不需要很大。真正目标是：

- 能检索到正确片段；
- 片段能支持答案；
- 没有答案时能够拒答。

9.3 资料格式

优先级：

1. Markdown；
2. TXT；
3. 排版规范的 DOCX；
4. 文本型 PDF；
5. 扫描 PDF；
6. 手机拍摄图片。

尽量先把教材内容整理成结构化 Markdown，而不是直接扔入质量很差的扫描 PDF。

示例：

五年级数学：通分

定义

把异分母分数分别化成和原来分数相等的同分母分数，叫作通分。

方法

1. 找到两个分母的最小公倍数；
2. 将最小公倍数作为公分母；
3. 根据分数的基本性质调整分子；
4. 检查所得分数是否与原分数相等。

例题

将 $\frac{2}{3}$ 和 $\frac{5}{8}$ 通分。

3 和 8 的最小公倍数是 24。

$$\frac{2}{3} = \frac{16}{24}$$

$$\frac{5}{8} = \frac{15}{24}$$

常见错误

- 只改变分母，没有同时改变分子；
- 公分母不是两个原分母的公倍数；
- 将通分误认为约分。

9.4 元数据

每份文档至少设置：

```
title
source
grade
subject
unit
document_type
version
status
updated_at
reviewer
```

例题：

```
{
  "title": "五年级数学：通分",
```

```
"source": "教师整理",
"grade": "五年级",
"subject": "数学",
"unit": "分数",
"document_type": "knowledge_note",
"version": "1.0",
"status": "active",
"updated_at": "2026-07-14",
"reviewer": "项目指导教师"
}
```

9.5 文档切分

起始设置可以使用：

- 每段约 300～600 个中文字符；
- 重叠 50～100 字；
- Top K 为 3～5；
- 先使用混合检索；
- 有条件时增加重排序。

但不同资料应采用不同切分方式：

文档类型	切分方法
教材知识点	按标题和小节
例题	一题一段
中心规定	一条规则一段
FAQ	一问一答
课程表	整张表或按日期
联系方式	一个人或一类岗位一段

9.6 检索 Prompt

请只根据“知识检索结果”回答用户问题。

要求：

1. 不得补充检索资料中没有的关键事实；
2. 优先使用符合用户年级和学科的资料；
3. 先用儿童能够理解的语言说明；
4. 学科题目采用分步骤提示；
5. 检索结果不足时明确说明资料库暂无可靠答案；
6. 不得伪造书名、页码、教师姓名或活动时间；

7. 在回答结尾列出使用到的资料标题；
8. 用户要求只给提示时，不直接给完整答案。

10. 第六阶段：拍照识题

10.1 最简实现

```
用户选择图片
↓
前端上传 Worker
↓
Worker 上传 Dify Files API
↓
获得 upload_file_id
↓
调用 chat-messages
↓
files 使用 local_file
```

Dify 的聊天 API 支持在消息中附带文件；对于本地文件，需要先上传并取得文件 ID，再将其作为 `upload_file_id` 传入聊天请求。

10.2 前端文件限制

第一版限制：

- JPEG、PNG、WebP；
- 单张图片；
- 最大 5 MB；
- 上传前压缩；
- 显示删除按钮；
- 上传失败可重试。

10.3 不要直接永久保存儿童照片

题目图片完成识别后：

- 不在前端公开地址长期保存；
- 日志不记录图片内容；
- 不用于人脸识别；
- 不默认纳入训练；
- 设置合理删除策略。

第一版甚至可以明确要求：

请只拍摄题目，不要拍摄人脸、姓名和个人信息。

11. 第七阶段：微信小程序壳

11.1 开发准备

需要：

1. 微信小程序账号；
2. 小程序 AppID；
3. 微信开发者工具；
4. 已部署的 HTTPS 网站；
5. 可配置的业务域名；
6. 根据上线方式完成相应平台要求。

小程序平台规则可能调整，正式上线前应在微信公众平台后台核对当前主体、备案、类目、内容审核、业务域名和隐私指引要求。

11.2 最小页面

`miniprogram/pages/index/index.wxml`：

```
<web-view src="{{webUrl}}"></web-view>
```

`index.ts`：

```
Page({
  data: {
    webUrl: "https://ai.example.com"
  }
});
```

`app.json`：

```
{
  "pages": [
    "pages/index/index"
  ],
  "window": {
    "navigationBarTitleText": "申望 AI 助教",
    "navigationBarBackgroundColor": "#ffffff",
    "navigationBarTextStyle": "black"
  }
}
```


11.3 建议保留网页直接入口

即使小程序上线，也应保留：

```
https://ai.example.com
```

原因：

- 小程序审核延期时仍可展示；
- 老师可以直接发送网页二维码；
- 电脑答辩时可以打开桌面版；
- 故障排查更方便。

11.4 WebView 方案的限制

该方案适合第一版，但存在：

- 网页与小程序之间通信能力有限；
- 微信原生登录接入更复杂；
- 语音、订阅消息等能力不如原生页面自然；
- 页面体验取决于网页加载速度；
- 域名及审核要求不能绕过；
- 外部文件下载和跳转可能受限制。

因此长期路线可以是：

```
第一版：WebView 壳  
第二版：首页和登录原生化  
第三版：聊天页使用 uni-app / 原生小程序重写
```

12. 部署教程

12.1 部署 Worker

在 `worker` 目录：

```
npx wrangler login  
npx wrangler secret put DIFY_API_KEY  
npm run deploy
```

获得地址：

```
https://shenwang-ai-proxy.<账户>.workers.dev
```

更新生产环境允许来源：

```
{
  "vars": {
    "DIFY_BASE_URL": "https://api.dify.ai/v1",
    "ALLOWED_ORIGIN": "https://ai.example.com"
  }
}
```

12.2 部署 React 到 Cloudflare Pages

将项目推送 GitHub：

```
git init
git add .
git commit -m "Initial AI tutor project"
git branch -M main
git remote add origin <GitHub仓库地址>
git push -u origin main
```

Cloudflare Pages 配置：

```
Framework preset: Vite
Root directory: web
Build command: npm run build
Output directory: dist
```

环境变量：

```
VITE_API_BASE_URL=
https://shenwang-ai-proxy.<账户>.workers.dev
```

部署成功后绑定自定义域名：

```
ai.example.com
```

12.3 环境划分

建议至少分：

环境	用途
Local	本地开发
Preview	队内测试
Production	正式演示

Cloudflare Worker Secret 和 Dify API Key 不应在测试截图、答辩 PPT 或 GitHub 中出现。

13. 安全设计

13.1 必须实现

API Key 保护

只保存在 Worker Secret 中。

CORS 白名单

仅允许正式网站和本地开发地址。

输入长度限制

例如：

- 文本最多 2000 字；
- 图片最大 5 MB；
- 一次最多一张图片。

限流

初期可以限制：

同一匿名用户：
每分钟最多 5 次；
每天最多 50 次。

若没有登录系统，可通过浏览器生成匿名 UUID，再结合 Worker 端简单限流。

输出安全

高风险内容直接进入固定流程，不让普通 Agent 自由回答。

日志脱敏

日志只记录：

请求时间
匿名用户 ID
耗时
成功或失败
模型用量
意图类别

尽量不要记录：

- 完整儿童对话；
- 图片；
- 姓名；
- 电话；
- 家庭情况；
- 住址。

13.2 高风险问题流程

检测到高风险
↓
停止普通答疑
↓
显示固定关怀提示
↓
提示联系现场老师或可信任成年人
↓
提供“向老师求助”按钮
↓
教师端形成待处理记录

固定回复示例：

你说的情况可能需要老师或其他可信任的成年人
马上帮助你。请现在联系现场老师，不要独自处理。

如果你正处于现实危险中，请立即离开危险位置，
向附近可信任的成年人求助。

不要让模型：

- 作出医疗或心理诊断；
- 承诺绝对保密；
- 独立调查家庭问题；
- 指导儿童隐瞒风险；
- 给出危险操作步骤。

14. 教师管理端

第一版不必开发完整后台，可以直接使用 Dify 后台管理知识库。

另外制作一个展示型统计页面：

今日提问数

本周活跃人数

高频知识点

最常见错误

未解决问题

用户反馈

平均响应时间

数据来源分阶段：

第一阶段

使用明确标注的演示数据。

第二阶段

Worker 记录匿名统计，存入：

- Cloudflare D1；
- 或 Supabase；
- 或简单后台数据库。

第三阶段

提供教师登录和知识审核。

不要把虚构数据包装成真实运营效果。

15. 测试方案

15.1 基础功能测试

测试项	预期结果
普通文本提问	正常流式回答
连续追问	保持上下文
新建会话	conversation_id 被清除
超长输入	前后端均拦截

测试项	预期结果
网络断开	显示重试
Dify 超时	显示服务繁忙
图片过大	上传前提示
非图片文件	拒绝上传
重复点击发送	不重复创建请求

15.2 RAG 测试

建立约 100 个问题：

40 个知识库中有明确答案的问题
 20 个口语化改写问题
 10 个多条件问题
 10 个知识库中没有答案的问题
 10 个容易检索错资料的问题
 10 个恶意或诱导问题

记录：

字段	内容
问题	用户输入
期望资料	应检索的文档
实际资料	系统检索结果
答案正确	是/否
引用支持	是/否
应拒答	是/否
响应时间	秒
备注	错误原因

15.3 演示前固定测试题

准备 8~10 个稳定问题：

1. 中心今天有什么课程？
2. 什么叫通分？
3. 给我一点提示，不要直接告诉我答案。
4. 上传一道清晰的数学题。
5. 生成一道类似题。
6. 提问知识库中不存在的信息。

7. 输入一个明显敏感的问题。
8. 断网后重新连接。
9. 新建会话。
10. 查看资料来源。

答辩现场尽量使用测试过的问题，不要完全依赖随机发挥。

16. 具体开发顺序

里程碑一：可聊天网页

完成：

- React 项目；
- 手机聊天界面；
- Worker 代理；
- Dify 简单 Chatflow；
- 流式输出；
- 会话连续性。

验收标准：

浏览器中可以连续提问，API Key 不出现在前端。

里程碑二：轻量知识库

完成：

- 中心知识库；
- 一个数学单元；
- 意图分类；
- 知识检索；
- 无答案拒答；
- 来源展示。

验收标准：

20 个固定问题中，至少大部分能够检索正确资料。

里程碑三：增强展示

完成：

- 拍照识题；
- 分步提示；
- 快捷问题；
- 动画；
- 加载状态；

- 错误重试；
- 基础统计页面。

里程碑四：微信小程序

完成：

- 注册小程序；
- 配置域名；
- 创建 WebView 壳；
- 真机测试；
- 隐私说明；
- 提交审核或内部体验版。

里程碑五：现场试用

完成：

- 教师审核知识库；
- 真实网络环境测试；
- 儿童试用反馈；
- 修改无法理解的表达；
- 清理日志和测试数据。

17. 团队分工

如果有四个人：

人员	工作
A	React 前端与视觉
B	Dify 工作流、Prompt、模型
C	资料整理、知识库、测试集
D	Worker、部署、小程序、文档

如果只有两个人：

人员	工作
A	前端、小程序、视觉
B	Dify、知识库、Worker、部署

你本人可以重点负责：

- 产品设计；
- 前端页面；

- Dify 可视化流程；
- 项目包装和答辩。

不需要深入研究模型训练或向量数据库底层。

18. 最终技术栈

前端：

React + TypeScript + Vite

Tailwind CSS

Zustand

react-markdown + KaTeX

Motion

Lucide

托管：

Cloudflare Pages

代理后端：

Cloudflare Workers

TypeScript

Fetch API

Worker Secrets

AI：

Dify Cloud

Chatflow

知识检索节点

条件分支

LLM 节点

可选 Agent/Tool 节点

知识库：

中心资料库

五年级数学示范库

微信端：

微信小程序

WebView

微信开发者工具

协作：

GitHub

Figma

Bruno/Postman

Markdown

19. 第一版明确不做的功能

为了防止项目失控，暂不开发：

- 全学科、全年级知识库；
- 复杂账户和积分系统；
- 家长端；
- 自动批改所有类型作业；
- 视频生成；
- 语音数字人；
- 多 Agent 辩论；
- 模型微调；
- 本地部署大模型；
- 原生 Android APK；
- 全自动心理干预；
- 自动向教师发送未经确认的严重预警；
- 复杂推荐算法。

第一版做成一个稳定的“垂直 AI 答疑产品”，比做十几个不稳定功能更有展示价值。

20. 项目最终包装

项目名称示例：

望学伴——基于 Agentic RAG 的儿童课后智能助教

技术描述：

项目采用移动端响应式 Web 与微信小程序双入口架构，以 Cloudflare Worker 作为安全代理层，通过 Dify Chatflow 编排意图识别、知识库检索、多模态题目理解和回答生成能力。系统根据用户问题自动路由至教材知识库、中心信息知识库或安全处理流程，并以流式方式返回具有资料依据的分步骤回答。

创新点不要写成“使用了大模型 API”，而应写：

1. 面向关爱中心实际资料构建本地化知识库；
 2. 根据问题类型动态调用不同知识源；
 3. 将完整答案改造成分步启发式回答；
 4. 提供微信内低门槛入口；
 5. 针对儿童场景设计风险分流；
 6. 通过高频问题统计辅助教师优化教学。
-

21. 立即执行清单

第一轮实际操作顺序：

1. 注册 Dify 并接入一个模型
2. 创建最简单的 Chatflow
3. 用 curl 测试 /chat-messages
4. 创建 Cloudflare Worker
5. 将 Dify Key 写入 Worker Secret
6. 测试 Worker 流式转发
7. 创建 React + Vite 项目
8. 完成聊天页面
9. 接入 Worker SSE
10. 导入 10 份测试资料
11. 加入知识检索节点
12. 建立 20 个固定测试问题
13. 部署 Pages 和 Worker
14. 绑定 HTTPS 域名
15. 创建微信小程序 WebView 壳
16. 真机测试

完成第 9 步时，项目已经有一个可演示的 AI 聊天产品；完成第 12 步时，项目才真正具备垂直场景价值；小程序应放到最后接入，避免平台配置问题阻塞核心开发。